AWS CLOUD DEPLOYMENT PROJECT

Deploying a Secure Flask Web Application on AWS

EC2 (Public Subnet) • RDS MySQL (Private Subnet) • S3 • IAM Roles • Security Groups

Student Project Guide

Student Name: _____

Batch / Cohort: _____

Start Date: _____ Submission Date: _____

Estimated Duration: 6–8 hours (can be split across 2 lab sessions)

Source Repository: https://github.com/gops601/pythoncodeAWS.git

Table of Contents

Table of Contents 2
1. Project Overview 4
1.1 What the Starter Code Does 4
2. Learning Objectives 4
3. Target Architecture 4
3.1 Component Summary 5
4. Prerequisites 5
5. Step-by-Step Tasks 7
Part A — Networking: VPC, Subnets, Routing 7
A1. Create the VPC 7
A2. Create the Public Subnet 7
A3. Create the Private Subnets 7
A4. Internet Gateway & Route Tables 7
A5. (Optional) NAT Gateway 7
Part B — Security Groups 7
B1. web-sg (attach to EC2) 7
B2. db-sg (attach to RDS) 7
Part C — IAM Role for EC2 → S3 Access 8
C1. Create an IAM Policy 8
C2. Create the IAM Role 8
C3. Attach the Role to EC2 8
Part D — S3 Bucket 8
D1. Create the Bucket 8
D2. Decide on Public Access 8
Part E — RDS MySQL (Private Subnet) 9
E1. Create a DB Subnet Group 9
E2. Launch the RDS Instance 9
E3. Create the Table 9
Part F — EC2 Instance & Deploying the App 9
F1. Launch the EC2 Instance 9
F2. Connect & Install Dependencies 10
F3. Fix the Hardcoded Credentials (Required) 10
F4. Run the App (systemd + gunicorn) 10
Part G — Testing & Validation 11

6. Security Checklist 12
7. Deliverables & Submission 12
8. Grading Rubric 12
9. Troubleshooting Guide 13
10. Bonus Challenges (Optional) 13
Suggested Time Allocation ..... **Error! Bookmark not defined.**
Common Student Mistakes to Watch For ..... **Error! Bookmark not defined.**
Discussion Prompts (for live session wrap-up) ..... **Error! Bookmark not defined.**

1. Project Overview

In this project you will take a working — but insecure — Flask web application and deploy it onto AWS using production-style architecture. The application is a simple "Student Registration" form: a visitor enters their name, email, and course, uploads a photo, and the app stores the photo in Amazon S3 and the registration details (including the photo's S3 URL) in a MySQL database.

The starter code lives in the instructor-provided repository. You will clone it onto your own EC2 instance, fix a security problem in it (hardcoded database credentials), and connect it to AWS-managed infrastructure that you design yourself.

Repository: https://github.com/gops601/pythoncodeAWS.git

This is not just a "click buttons in AWS" exercise. The core skill being assessed is designing network and identity boundaries correctly: what belongs on the public internet, what must never be reachable from the internet, and how two AWS resources should be allowed to talk to each other without either one holding a long-lived secret.

1.1 What the Starter Code Does

File	Purpose
app.py	Flask app with two routes: / (renders the form) and /register (POST — uploads photo to S3, inserts a row into MySQL).
templates/index.html	Plain HTML registration form (name, email, course, photo file input).
requirements.txt	Python dependencies: Flask, gunicorn, boto3, PyMySQL, etc.

Look closely at app.py before you start. You will notice the database host, username, and password are hardcoded directly in the source file, and the S3 bucket name is hardcoded too. Both of these are things a real engineering team would reject in code review. Part of your task is to fix this (see Part F, Task F5).

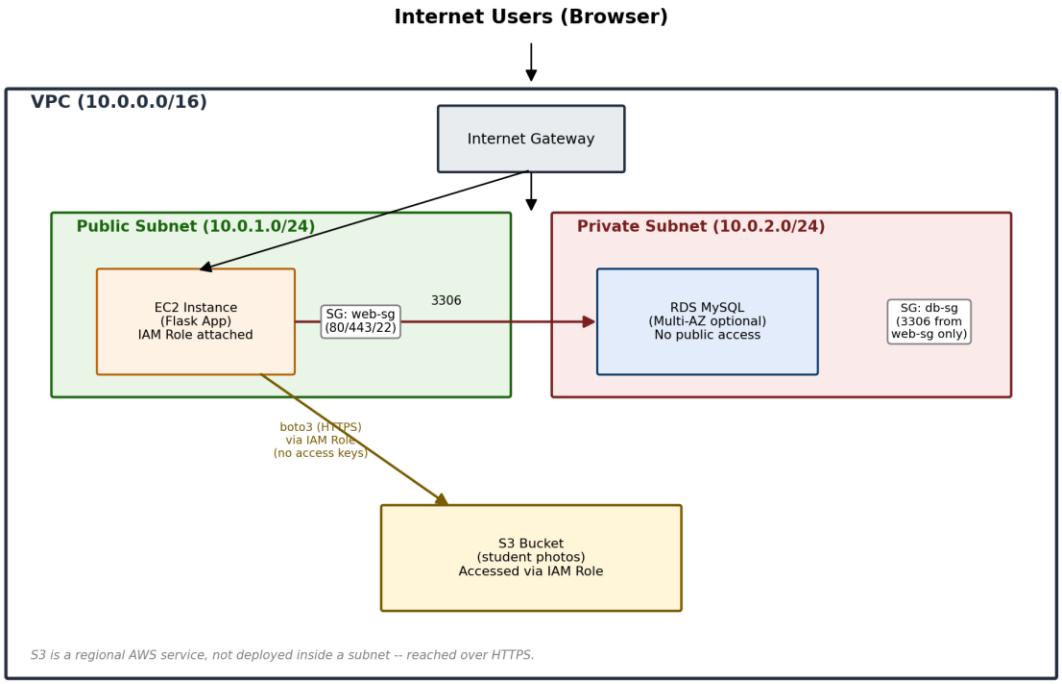
2. Learning Objectives

By the end of this project, you should be able to:

- Design a custom VPC with properly separated public and private subnets.
- Configure Security Groups using the principle of least privilege (referencing other security groups instead of open CIDR ranges).
- Create and attach an IAM role so an EC2 instance can call S3 without any hardcoded access keys.
- Deploy a MySQL database on Amazon RDS inside a private subnet with no public accessibility.
- Deploy and run a Python Flask application on an EC2 instance in a public subnet.
- Externalize configuration (DB credentials, bucket names) instead of hardcoding secrets in source code.
- Validate a security posture by proving that private resources really are unreachable from outside the VPC.

3. Target Architecture

You will build the architecture shown below. The EC2 instance is the only resource reachable from the internet. The database sits in a private subnet with no route to an Internet Gateway. S3 is a regional AWS service (not something you place "inside" a subnet) — the EC2 instance reaches it over HTTPS using temporary credentials supplied by its attached IAM role.



3.1 Component Summary

Component	Location	Reachable From Internet?	Access Control
EC2 (Flask app)	Public subnet	Yes — HTTP/HTTPS (and SSH from your IP only)	Security group: web-sg
RDS MySQL	Private subnet(s)	No	Security group: db-sg (inbound 3306 only from web-sg)
S3 bucket	Regional service (outside VPC)	Only the specific object path, if you choose to allow public reads	IAM role (EC2 → S3) + bucket policy

4. Prerequisites

- An AWS account (Free Tier is sufficient) with permissions to create VPCs, EC2, RDS, S3, and IAM resources.
- Basic comfort with the Linux command line and SSH.
- Git installed locally, and a GitHub account (you will fork the starter repo).

- Python 3.9+ and pip installed locally (for optional local testing).
- A text editor / IDE (VS Code recommended).
- Access to the starter repository: https://github.com/gops601/pythoncodeAWS.git

5. Step-by-Step Tasks

Work through Parts A–G in order. Each part builds on the previous one. Take screenshots as you go — you will need them for submission (Section 7).

Part A — Networking: VPC, Subnets, Routing

A1. Create the VPC

Create a VPC with CIDR block 10.0.0.0/16. Give it a clear name, e.g. student-app-vpc.

A2. Create the Public Subnet

Create one subnet, 10.0.1.0/24, in Availability Zone "a". Name it public-subnet-1. This is where the EC2 instance will live.

A3. Create the Private Subnets

Create two private subnets in two different Availability Zones — for example 10.0.2.0/24 in AZ "a" and 10.0.3.0/24 in AZ "b". Name them private-subnet-1 and private-subnet-2.

Why two private subnets? RDS requires a "DB Subnet Group" spanning at least two Availability Zones, even if you only ever use one AZ for the actual database instance. Create both now to avoid rework later.

A4. Internet Gateway & Route Tables

- Create an Internet Gateway and attach it to your VPC.
- Create (or edit the main) route table for the public subnet: add a route 0.0.0.0/0 → your Internet Gateway. Associate this route table with public-subnet-1.
- Leave the private subnets on a route table that has no route to the Internet Gateway (the default local-only route is fine). This is what makes them "private."

A5. (Optional) NAT Gateway

If you want the private subnets to be able to reach the internet for outbound-only traffic (e.g. OS patching), create a NAT Gateway in the public subnet and add a 0.0.0.0/0 route pointing to it from the private route table. This is not required for the app to function and is a good bonus task (see Section 9).

Part B — Security Groups

B1. web-sg (attach to EC2)

Direction	Type	Port	Source/Destination
Inbound	SSH	22	Your IP only (My IP), never 0.0.0.0/0
Inbound	HTTP	80 (or 5000 while testing)	0.0.0.0/0
Outbound	All traffic	All	0.0.0.0/0 (default)

B2. db-sg (attach to RDS)

Direction	Type	Port	Source/Destination
Inbound	MySQL/Aurora	3306	web-sg (select the security group itself, not a CIDR range)
Outbound	All traffic	All	0.0.0.0/0 (default)

Key idea: the database's inbound rule should reference the web-sg security group as its source, not an IP range or "0.0.0.0/0". This means only instances that belong to web-sg can ever reach port 3306, regardless of what IP address they're given.

Part C — IAM Role for EC2 → S3 Access

C1. Create an IAM Policy

Create a customer-managed IAM policy that allows only the actions the app needs, scoped to only your bucket:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
    }
  ]
}
```

C2. Create the IAM Role

Create an IAM role named ec2-s3-role. Choose "EC2" as the trusted entity (this sets the trust policy so only the EC2 service can assume it). Attach the policy you created in C1.

C3. Attach the Role to EC2

Attach ec2-s3-role to your EC2 instance — either at launch time ("IAM instance profile" field) or afterwards via Actions → Security → Modify IAM role.

Do not create an IAM user with access keys for this. The whole point of the instance role is that boto3 on the EC2 instance automatically obtains short-lived, auto-rotating credentials — there should be no AWS access key or secret key anywhere in your code, your .env file, or your git history.

Part D — S3 Bucket

D1. Create the Bucket

Create an S3 bucket with a globally-unique name (e.g. student-photos-yourname-2026).

D2. Decide on Public Access

The starter code builds photo URLs in the form https://BUCKET.s3.amazonaws.com/filename, which only works if that object is publicly readable. You have two valid options — pick one and be ready to justify it in your write-up:

- **Option 1 (matches the starter code):** keep "Block all public access" OFF only for reads, and attach a bucket policy that allows s3:GetObject to everyone but does NOT allow s3:ListBucket or any write actions publicly. Uploads still only happen through the IAM role.
- **Option 2 (more secure, recommended stretch goal):** keep the bucket fully private and modify the app to generate a time-limited pre-signed URL (s3_client.generate_presigned_url(...)) instead of a public URL. See Section 9, Bonus Challenges.

Whichever option you choose, "Block all public access" being fully OFF with no bucket policy restriction is not acceptable — that would allow anyone to list and read the entire bucket.

Part E — RDS MySQL (Private Subnet)

E1. Create a DB Subnet Group

Create a DB Subnet Group containing private-subnet-1 and private-subnet-2 (from Task A3).

E2. Launch the RDS Instance

Setting	Value
Engine	MySQL 8.x
Instance class	db.t3.micro (Free Tier eligible)
Public access	No
VPC security group	db-sg (from Task B2)
DB subnet group	The one created in E1
Initial database name	studentdb
Master username / password	Choose your own — do not reuse the ones from the starter code

E3. Create the Table

Once the instance is available, note its endpoint hostname. Because RDS is not publicly reachable, you must connect to it from inside the VPC — SSH into your EC2 instance first, then connect to MySQL from there:

```
mysql -h <your-rds-endpoint> -u <username> -p studentdb
```

Then create the table the app expects:

```
CREATE TABLE students (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  course VARCHAR(100),
  photo_url VARCHAR(500)
);
```

Part F — EC2 Instance & Deploying the App

F1. Launch the EC2 Instance

Setting	Value
AMI	Amazon Linux 2023 (or Ubuntu 22.04 — adjust commands accordingly)
Subnet	public-subnet-1
Auto-assign public IP	Enable
Security group	web-sg (from Task B1)
IAM instance profile	ec2-s3-role (from Task C2)
Key pair	Create or reuse one you control

F2. Connect & Install Dependencies

```
ssh -i your-key.pem ec2-user@<EC2-PUBLIC-IP>
sudo yum update -y
sudo yum install -y python3-pip git mysql
git clone https://github.com/gops601/pythoncodeAWS.git
cd pythoncodeAWS
pip3 install -r requirements.txt
```

F3. Fix the Hardcoded Credentials (Required)

Before running the app, edit app.py so the database connection and bucket name come from environment variables instead of being hardcoded. Replace the connection block with something like:

```
import os

bucket_name = os.environ.get("S3_BUCKET_NAME")

db = pymysql.connect(
    host=os.environ.get("DB_HOST"),
    user=os.environ.get("DB_USER"),
    password=os.environ.get("DB_PASSWORD"),
    database=os.environ.get("DB_NAME", "studentdb")
)
```

Then set those variables on the instance — for a quick test you can export them in the shell, but for anything long-running use a systemd unit (Task F4) so they don't need to be re-exported every reboot:

```
export DB_HOST=<your-rds-endpoint>
export DB_USER=<your-username>
export DB_PASSWORD=<your-password>
export DB_NAME=studentdb
export S3_BUCKET_NAME=<your-bucket-name>
```

Note: *boto3.client('s3') does not need any credentials passed to it — it will automatically discover the credentials supplied by your IAM instance role. Do not add aws_access_key_id / aws_secret_access_key anywhere.*

F4. Run the App (systemd + gunicorn)

For a quick manual test:

```
python3 app.py
# visit http://<EC2-PUBLIC-IP>:5000
```

For something that survives logout and reboots, run it with gunicorn under systemd. Create /etc/systemd/system/flaskapp.service:

```
[Unit]
Description=Flask Student Registration App
After=network.target

[Service]
User=ec2-user
WorkingDirectory=/home/ec2-user/pythoncodeAWS
Environment="DB_HOST=<your-rds-endpoint>"
Environment="DB_USER=<your-username>"
Environment="DB_PASSWORD=<your-password>"
Environment="DB_NAME=studentdb"
Environment="S3_BUCKET_NAME=<your-bucket-name>"
ExecStart=/usr/local/bin/gunicorn -w 3 -b 0.0.0.0:80 app:app
Restart=always

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now flaskapp
sudo systemctl status flaskapp
```

Note: *binding to port 80 requires root privileges — running gunicorn via systemd as shown (or with an authbind/CAP_NET_BIND_SERVICE setup) handles this. Alternatively bind to a high port like 8000 and put Nginx in front of it (see Bonus Challenges).*

Part G — Testing & Validation

- Open http://<EC2-PUBLIC-IP> in a browser, fill in the registration form with a test photo, and submit it.
- SSH into EC2 and query MySQL to confirm the row was inserted: SELECT * FROM students;
- Check the S3 bucket in the console (or aws s3 ls s3://your-bucket-name) to confirm the photo was uploaded.
- From your own laptop (not the EC2 instance), try to connect to the RDS endpoint on port 3306. It should time out — this proves the private subnet and security group are actually working.
- Try to open https://your-bucket.s3.amazonaws.com/ (no filename) in a browser. It should be denied (Access Denied / 403) — this proves you haven't accidentally allowed public bucket listing.

6. Security Checklist

Tick off each item before you submit. If any item fails, fix it — these map directly to the grading rubric in Section 8.

- No AWS access keys, secret keys, or DB passwords appear anywhere in app.py or in your git history.
- RDS instance has "Publicly accessible" set to No.
- db-sg only allows inbound 3306 from web-sg (not from 0.0.0.0/0 or a broad CIDR).
- web-sg only allows inbound SSH (22) from your own IP, not 0.0.0.0/0.
- EC2 reaches S3 using an attached IAM role — no hardcoded credentials in boto3 calls.
- The IAM policy attached to the role is scoped to your specific bucket ARN, not "Resource": "*".
- S3 bucket does not allow public s3:ListBucket, even if individual object reads are public.
- Private subnets have no route to an Internet Gateway.
- Environment variables / secrets are not committed to your GitHub fork (check your .gitignore).

7. Deliverables & Submission

Submit the following as a single PDF or shared folder link:

- A diagram of the architecture you actually built (hand-drawn, draw.io, or a screenshot annotated with labels is fine).
- Screenshots: VPC + subnet list, both security groups (inbound rules visible), IAM role + attached policy, RDS instance detail page showing "Publicly accessible: No", EC2 instance detail page showing the attached IAM role.
- A screenshot of the working registration form and a successful submission.
- A screenshot of the inserted row in MySQL and the uploaded object in S3.
- A link to your own GitHub fork containing the fixed app.py (environment-variable version — with no real secrets committed).
- A short write-up (roughly half a page) explaining: which S3 access option you chose (D2) and why, and one thing you'd improve if you had more time.
- The completed Security Checklist from Section 6.

8. Grading Rubric

Criteria	Points	What's Assessed
Networking design	15	Correct VPC/subnet CIDR layout, IGW attached only to public route table, private subnets have no internet route.
Security groups	15	Least-privilege rules; db-sg references web-sg, not an open CIDR; SSH restricted.
IAM role usage	20	EC2 uses an instance role (no access keys anywhere); policy scoped to one bucket, not "*".

Criteria	Points	What's Assessed
RDS deployment	20	Instance is in the private subnet group, not publicly accessible, and the app can read/write it correctly.
S3 integration & bucket security	15	Upload works; public access is deliberately scoped (Option 1 or 2 from Task D2), not wide open.
End-to-end functionality	10	Form submission actually stores a row in RDS and a photo in S3.
Documentation & write-up	5	Checklist completed; write-up shows understanding of the security trade-offs made.
Total	100	

9. Troubleshooting Guide

Symptom	Likely Cause / Fix
"Can't connect to MySQL server" from EC2	Check db-sg inbound source is web-sg (not a CIDR); double-check the RDS endpoint hostname; confirm EC2 and RDS are in the same VPC.
App can't upload to S3 ("Access Denied")	Confirm the IAM role is actually attached to the instance (Instance → Security tab); confirm the policy's Resource ARN matches your bucket name exactly.
Webpage doesn't load at all	Confirm web-sg allows inbound on the port you're using (80 or 5000); confirm the app/gunicorn process is actually running (systemctl status flaskapp); confirm the public subnet's route table has a 0.0.0.0/0 route to the Internet Gateway.
git clone fails on EC2	The public subnet needs a route to the Internet Gateway and the instance needs a public IP — recheck Task A4 and the "auto-assign public IP" setting.
RDS creation stuck on "DB subnet group" error	You need at least two subnets in two different Availability Zones in the subnet group, even for a single-AZ database (Task A3).

10. Bonus Challenges (Optional)

- Replace the systemd + gunicorn setup with gunicorn behind an Nginx reverse proxy, and serve over HTTPS using a self-signed cert or an ACM certificate behind an Application Load Balancer.
- Add a NAT Gateway so private subnets have outbound-only internet access, and use it to run OS patch updates on a private instance.
- Replace the public S3 URL approach with pre-signed URLs (time-limited access), and keep "Block all public access" fully ON.

- Move DB credentials out of systemd environment variables and into AWS Secrets Manager, fetched at app startup via boto3.
- Put the EC2 instance behind an Application Load Balancer with an Auto Scaling Group (min 1, max 2) for basic high availability.
- Rebuild the entire environment as code using Terraform or AWS CloudFormation instead of the console.